

Exercise

Scenario Food delivery platform

You are building part of a food delivery platform like Uber Eats or Deliveroo.

Part 1 Core domain model

1 Abstract class MenuItem

Fields

- id int
- name String
- price double

Constructors

- Constructor with id and name
- Constructor with id, name, and price
- Use **constructor chaining** so only one constructor sets fields

Methods

- double getPrice()
- String getDescription() abstract

2 Interface Discountable

Method

double applyDiscount(double percentage)

Part 2 Concrete menu items

3 FoodItem class

Represents main meals that can be discounted.

Extends MenuItem

Implements Discountable

Extra field: **isVegetarian boolean**

Example food names to use for testing

- Margherita Pizza
- Chicken Burger
- Veggie Wrap

Requirements

- Implement getDescription using name and vegetarian status
- Implement applyDiscount by reducing price
- Use **super** in the constructor

4 SideItem class

Represents items that **cannot be discounted**.

Extends MenuItem

Does NOT implement Discountable

Example side item names to use for testing

- Garlic Bread
- French Fries

Requirement

- Implement getDescription
- No discount logic allowed

5 DrinkItem class

Extends MenuItem

Extra field: **sizeInML** int

Example drink names to use for testing:

- Cola
- Orange Juice
- Mineral Water
- Iced Coffee

Part 3 Order and ArrayList

Create a class Order.

Field: **ArrayList<MenuItem> items**

Methods

- void addItem(MenuItem item)
- double calculateTotalPrice()
- void printOrderItems()

Part 4 Stream API tasks

Methods

- Return vegetarian food items
List<FoodItem> getVegetarianItems()
- Total price of items above a given price
double totalPriceAbove(double price)
- Count drinks larger than 500ml
long countLargeDrinks()
- Return names of all drink items
List<String> getDrinkNames()

Part 5 - class Main

1 Create the following items

- FoodItem Margherita Pizza vegetarian
- FoodItem Chicken Burger not vegetarian
- SideItem Garlic Bread
- DrinkItem Cola 330ml
- DrinkItem Iced Coffee 600ml

2 Add all items to an Order

3 Apply discount to Margherita Pizza only

4 Print full order

5 Use streams to

- List vegetarian foods
- Count large drinks
- Show drink names
- Calculate totals

Questions

1. Why does SideItem not implement Discountable
2. What happens if you try to discount every MenuItem
3. Why is ArrayList<MenuItem> a good design choice
4. How does constructor chaining simplify constructors
5. Where does polymorphism appear in this design

```

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

abstract class MenuItem {

    protected int id;
    protected String name;
    protected double price;

    public MenuItem(int id, String name) {
        this(id, name, 0.0);
    }

    public MenuItem(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public abstract String getDescription();
}

interface Discountable {
    double applyDiscount(double percentage);
}

class FoodItem extends MenuItem implements Discountable {

    private boolean isVegetarian;

    public FoodItem(int id, String name, double price, boolean
                                                              isVegetarian) {
        super(id, name, price);
        this.isVegetarian = isVegetarian;
    }

    public boolean isVegetarian() {
        return isVegetarian;
    }

    @Override
    public double applyDiscount(double percentage) {
        double discount = price * (percentage / 100);
        price -= discount;
        return price;
    }

    @Override
    public String getDescription() {
        return name + (isVegetarian ? " (Vegetarian)" :
                                     " (Non vegetarian)");
    }
}

class SideItem extends MenuItem {

```

```

    public SideItem(int id, String name, double price) {
        super(id, name, price);
    }

    @Override
    public String getDescription() {
        return name + " (Side item)";
    }
}

class DrinkItem extends MenuItem {

    private int sizeInMl;

    public DrinkItem(int id, String name, double price, int sizeInMl) {
        super(id, name, price);
        this.sizeInMl = sizeInMl;
    }

    public int getSizeInMl() {
        return sizeInMl;
    }

    @Override
    public String getDescription() {
        return name + " " + sizeInMl + "ml";
    }
}

class Order {

    private ArrayList<MenuItem> items = new ArrayList<>();

    public void addItem(MenuItem item) {
        items.add(item);
    }

    public double calculateTotalPrice() {
        return items.stream()
            .mapToDouble(MenuItem::getPrice)
            .sum();
    }

    public void printOrderItems() {
        items.forEach(item ->
            System.out.println(item.getDescription() + " £" +
                item.getPrice())
        );
    }

    public List<FoodItem> getVegetarianItems() {
        return items.stream()
            .filter(item -> item instanceof FoodItem)
            .map(item -> (FoodItem) item)
            .filter(FoodItem::isVegetarian)
            .collect(Collectors.toList());
    }

    public double totalPriceAbove(double price) {
        return items.stream()
            .filter(item -> item.getPrice() > price)

```

```

        .mapToDouble(MenuItem::getPrice)
        .sum();
    }

    public long countLargeDrinks() {
        return items.stream()
            .filter(item -> item instanceof DrinkItem)
            .map(item -> (DrinkItem) item)
            .filter(drink -> drink.getSizeInMl() > 500)
            .count();
    }

    public List<String> getDrinkNames() {
        return items.stream()
            .filter(item -> item instanceof DrinkItem)
            .map(item -> item.name)
            .collect(Collectors.toList());
    }
}

public class Main {
    public static void main(String[] args) {

        FoodItem pizza = new FoodItem(1, "Margherita Pizza", 9.99, true);
        FoodItem burger = new FoodItem(2, "Chicken Burger", 8.49, false);
        SideItem garlicBread = new SideItem(3, "Garlic Bread", 3.99);

        DrinkItem cola = new DrinkItem(4, "Cola", 1.99, 330);
        DrinkItem icedCoffee = new DrinkItem(5, "Iced Coffee", 3.49, 600);

        Order order = new Order();
        order.addItem(pizza);
        order.addItem(burger);
        order.addItem(garlicBread);
        order.addItem(colas);
        order.addItem(icedCoffee);

        pizza.applyDiscount(10);

        System.out.println("Order items");
        order.printOrderItems();

        System.out.println("\nVegetarian items");
        order.getVegetarianItems()
            .forEach(item -> System.out.println(item.getDescription()));

        System.out.println();
        System.out.println("Large drinks count: " +
            order.countLargeDrinks());
        System.out.println("Drink names: " + order.getDrinkNames());
        System.out.println("Total price: £" + order.calculateTotalPrice());
    }
}

```